Chapter 4

RL Foundations for Language Models

Supervised fine-tuning (SFT) teaches a model to imitate demonstrations, but imitation has a ceiling: the model can never exceed the quality of its training data. Reinforcement learning breaks this barrier. By generating novel text, receiving reward feedback, and updating toward higher-reward behaviours, an RL-trained model can *discover* strategies that no human demonstrator wrote—producing outputs that are more helpful, more accurate, and better aligned with human preferences [280].

This is the mechanism behind every frontier model: GPT-4 [274], Claude, Llama-3 [118], and DeepSeek-R1 [72] all apply RL after SFT as the critical step that transforms a capable but unsteered model into an aligned assistant.

4.1 Two Paradigms for RL in LLMs

RL methods for language models fall into two broad paradigms, each suited to different goals:

Paradigm 1: Alignment via Human Preferences (RLHF/DPO). The original motivation for applying RL to LLMs was **alignment**—making models helpful, harmless, and honest. **Reinforcement Learning from Human Feedback (RLHF)** [57, 280, 442] trains a reward model from pairwise human judgments ("which response is better?") and then optimizes the policy to maximize that learned reward. **DPO** [302] simplifies this by eliminating the reward model entirely, converting preferences directly into a supervised loss. Both approaches produce aligned assistants that follow instructions and respect safety constraints.

Paradigm 2: Capability Enhancement via Verifiable Rewards (RLVR). More recently, RL has been used not just for alignment but for **teaching new capabilities**—particularly reasoning, mathematics, and code generation. Here the reward comes not from human preferences but from **verifiable outcomes**: did the model produce the correct answer? Did the code pass all tests? DeepSeek-R1 [72] demonstrated that GRPO with rule-based rewards (format correctness + answer accuracy) can train models to develop sophisticated chain-of-thought reasoning *without any human preference data*. This paradigm—RL from Verifiable Rewards (RLVR)—is now the dominant approach for building reasoning models and agentic systems.

The Shared Foundation

Despite their different goals, both paradigms share the same core machinery:

- A **policy** π_θ (the LLM) that generates text autoregressively
- A **reward signal** $r(x, y)$ (learned from preferences or computed from verification)
- A **KL constraint** against a reference policy to prevent degenerate solutions
- **Policy gradient optimization** (PPO or GRPO) to update the model toward higher reward

The chapters in this part develop each component in detail.

4.2 Text Generation as an MDP

The key insight that makes RL applicable to language models is recasting autoregressive generation as a Markov Decision Process:

The LLM-as-Agent Analogy

Think of the LLM as an **agent** writing a response one token at a time. At each step, it looks at everything written so far (the *state*), chooses the next word (the *action*), and the page grows by one token (the *transition*). When the response is complete, a judge scores it (the *reward*). The goal: learn a writing strategy (a *policy*) that consistently earns high scores.

Formally, the MDP for text generation is:

- **State** $s_t = (x, y_1, \ldots, y_{t-1})$: the prompt concatenated with all tokens generated so far.
- **Action** $a_t \in \{1, \ldots, |\mathcal{V}|\}$: choosing the next token from the vocabulary (32K–128K options).
- **Transition** $P(s_{t+1}|s_t, a_t)$: deterministic—just append the chosen token. No environment stochasticity.
- **Reward** r: typically given only at the end of generation (sparse). For RLHF: the reward model score. For RLVR: correctness of the final answer.
- **Policy** $\pi_\theta(a_t|s_t)$: the LLM's next-token probability distribution—exactly what the softmax output already computes.
- **Discount** $\gamma = 1.0$: episodes are finite (one response), so no discounting needed.

This mapping is powerful because the LLM *already is* a policy—its softmax output defines $\pi_\theta(a_t|s_t)$ for every state. We don't need to build a separate policy network; we just need to adjust the weights θ so the model assigns higher probability to token sequences that earn higher reward.

4.3 The RLHF Pipeline

The classic RLHF pipeline [280] consists of four stages:

1. **Supervised Fine-Tuning (SFT)**: Train a base model on high-quality demonstrations to produce a policy π_{SFT} that can follow instructions.
2. **Reward Model Training**: Collect human preference comparisons ($y_w \succ y_l$ for the same prompt) and train a reward model $R_\phi(x, y)$ using the Bradley-Terry objective.
3. **RL Optimization**: Use the reward model as a signal to optimize the policy via PPO or GRPO, subject to a KL constraint against π_{SFT}.
4. **Evaluation and Iteration**: Evaluate the aligned model, collect new failure cases, and iterate.

For RLVR (reasoning/agentic training), stages 1–2 are replaced: the SFT model is trained on reasoning traces, and the reward model is replaced by a verifier (e.g., checking mathematical correctness). Stage 3 remains the same—PPO or GRPO optimization against the reward signal.

How LLM RL Differs from Classical RL

The LLM setting differs from classical RL in important ways:

- **Deterministic transitions**: The "next state" is just the concatenation of previous tokens—no stochastic environment.
- **Sparse reward**: Feedback is typically given once at the end of generation (outcome reward) or at key steps (process reward).

- **Massive action space**: 32K–128K possible tokens at every step, but exploration is implicit via temperature sampling.
- **KL anchor**: LLM RL is constrained to stay close to the SFT policy, preventing reward hacking at the cost of reduced exploration.
- **No value function needed**: GRPO eliminates the critic network entirely, using group-relative normalization of rewards instead.

These differences explain why PPO and GRPO dominate over DQN-style approaches for LLMs.

4.4 Roadmap of This Part

The chapters ahead build the complete RL-for-LLMs toolkit:

1. **PPO** (Chapter 5) — The clipped surrogate objective, GAE for advantage estimation, the critic network, and the full RLHF training loop. The workhorse behind GPT-4 and Claude.
2. **DPO** (Chapter 6) — Bypassing RL entirely by converting preferences into a contrastive supervised loss. Simpler but less flexible than online RL.
3. **GRPO** (Chapter 7) — DeepSeek's critic-free algorithm that uses group-level reward normalization. The method behind DeepSeek-R1 and the dominant choice for reasoning model training.
4. **Preference optimization variants** (Chapter 8) — Online DPO, KTO, Best-of-N, and guidance on method selection.
5. **Reward modeling** (Chapter 9) — Bradley-Terry models, process vs. outcome rewards, rule-based rewards for RLVR, and multi-objective combinations.
6. **SFT best practices** (Chapter 10) — Sequence packing, chat templates, data mixing, and how SFT quality determines the RL ceiling.
7. **Systems engineering** (Chapter 11) — Distributed training at scale: parallelism strategies, generation–training decoupling, and infrastructure for hundreds of GPUs.